

GENERACIÓN DE NÚMEROS PSEUDOALEATORIOS

CONTENIDOS

1 **Introducción a la generación de números aleatorios**

2 **Un poco de historia**

3 **Definición de números pseudoaleatorios**

4 **Uso y aplicaciones de los números aleatorios**

5 **Generación de números pseudoaleatorios**

6 **Generador Congruencial Lineal**

i **Generador Congruencial Lineal Mixto**

* **Elección de parámetros adecuados**

ii **Generador Congruencial Lineal Multiplicativo**

* **Elección de parámetros adecuados**

7 **Generador por Método Del Cuadrado Medio**

8 **Bibliografía**

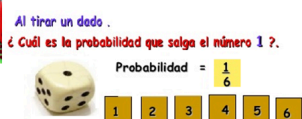
1.Introducción a la generación de números aleatorios

Un **NÚMERO ALEATORIO** es un número obtenido al azar.
Tiene la misma probabilidad de ser elegido y la elección de un número no depende de la elección del otro.

Los seres humanos vivimos en un medio aleatorio y nuestro comportamiento lo es también. Si deseamos predecir el comportamiento de un material, de un fenómeno climatológico, de un grupo humano, temas de ciberseguridad y generación de claves criptográficas, generación de mundos o mapas aleatorios en videos juegos o gráficos por computadora, entre múltiples ejemplos, podemos inferir a partir de datos estadísticos, números o variables aleatorias. Para lograr una mejor aproximación a la realidad nuestra herramienta predictiva debe funcionar de manera similar: **aleatoriamente**. De esa necesidad surgieron los modelos de simulación.

En la vida cotidiana se utilizan números aleatorios en situaciones tan dispares como pueden ser juegos de azar, animación por computadora, tests para localización de errores en chips, transmisión de datos desde un satélite, finanzas, estudios de modelos poblacionales, etc.

Los **números aleatorios** permiten a los modelos matemáticos representar la realidad.
En general cuando se requiere que determinados datos sean **impredecibles** se utilizan **números aleatorios**.



La lógica nos hace pensar que las personas somos generadores aleatorios imperfectos. Hay estudios que demuestran que existen tendencias claras en los humanos para la elaboración de secuencias sesgadas y están relacionadas con características personales, con los conocimientos o informaciones previas o con la edad.

El mundo real es muy raro que sea determinístico, al estudiar un sistema y sus influencias externas como la llegada de clientes a un banco y el comportamiento del sistema como el tiempo de servicio bancario, tienen un comportamiento aleatorio o probabilístico. En la construcción de un modelo de simulación bancario es necesario reproducir los efectos aleatorios del sistema, por lo cual es necesario incursionar en la generación de números aleatorios o pseudoaleatorios, como también la manera de poderlos generar y probar que dichos números tengan la aleatoriedad deseada.

El tiempo de servicio bancario varía dependiendo del tipo de transacciones que el cliente realice. Los datos históricos obtenidos en un banco tan solo se puede recopilar una muestra de una población desconocida, siendo estos datos limitados debido al tiempo y los costos que se requieren, En ocasiones no existe realmente el sistema (un banco que desea abrir una nueva sucursal) o la imposibilidad de obtener datos reales, o la obtención de grandes volúmenes de datos que originan un procesamiento lento o vuelve complejo el modelo a construir.

La obtención artificial de los datos del tiempo de servicio viene a simplificar el modelo, partiendo de una muestra significativa. Es importante incursionar en el estudio de la aleatoriedad y las pruebas estadísticas para garantizar que se cuenta con un generador confiable.

Cuando se entrena una **red neuronal**, los pesos de conexión entre las neuronas deben inicializarse antes de comenzar el proceso de aprendizaje. Si todos los pesos se inicializaran con el mismo valor (por ejemplo, todos ceros), la red aprendería muy mal o no aprendería en absoluto. Por eso, se utilizan **números aleatorios para dar valores iniciales distintos a cada peso**. Por ejemplo, un peso puede ser inicializado con un número aleatorio generado así: $w = \text{rand}(0,1)$, donde $\text{rand}(0, 1)$ genera un número aleatorio en el intervalo $[0,1]$. A modo de ejemplo, si el número es $w=0.7314$, dicho valor se usará como peso en una de las conexiones de la red neuronal. Es importante que la red comience con una diversidad de conexiones que permiten el aprendizaje. Si todos los pesos fueran iguales, las neuronas aprenderían lo mismo, y no habría diferenciación de funciones dentro de la red.

En **robótica**, los números aleatorios se usan en varias áreas, especialmente cuando el robot debe **tomar decisiones en entornos inciertos o simular comportamientos**. Si un robot autónomo debe explorar un terreno desconocido, para evitar quedar atrapado en ciclos o seguir trayectorias predecibles, puede incorporar movimientos aleatorios en su lógica de exploración. Un número aleatorio puede determinar, por ejemplo, la dirección en la que gira. Este valor aleatorio permite al robot decidir una nueva dirección de manera impredecible, lo que puede ser útil en algoritmos como exploración aleatoria (random walk), o como parte de estrategias más complejas como SLAM (localización y mapeo simultáneo) o algoritmos evolutivos.

Podemos aprovecharnos de situaciones reales para obtener una tabla de números aleatorios, como la lista de los números de Lotería Nacional premiados a lo largo de su historia, pues se caracterizan por que cada dígito tiene la misma probabilidad de ser elegido, y su elección es independiente de las demás extracciones.

Podemos encontrar números aleatorios por diferentes métodos:

Métodos manuales: lanzamiento de monedas, lanzamientos de dados, dispositivos mecánicos, dispositivos electrónicos.

Métodos de computación analógica: son métodos que dependen de ciertos procesos físicos aleatorios, por ejemplo, el comportamiento de una corriente eléctrica.

En esta generación se requiere conectar un dispositivo especializado a una computadora, ya que por medio de este dispositivo pueda proporcionar los números aleatorios. Esta técnica de generación es utilizada en simulaciones que requieren observar el ambiente físico y usualmente se utilizan interfaz analógica - digital las cual requieren de inversiones más costosas y tardan más tiempo en implementarse.

Métodos de computación digital: cuando se usa la computadora digital. Es la generación más utilizada en los procesos de simulación, mediante una relación de recurrencia. Una relación de recurrencia es una ecuación que define una secuencia recursiva donde cada término de la secuencia es definida como una función de los términos anteriores. Las relaciones de recurrencia son ampliamente utilizadas en la teoría de la probabilidad.

Tablas de bibliotecas: son números aleatorios que se han publicado; de los cuales podemos encontrar listas en los libros de probabilidad y tablas de matemáticas. Estos números fueron generados por alguno de los métodos de computación analógica.

Los números aleatorios son la base esencial de la simulación, ya que proporcionan la variabilidad necesaria para modelar situaciones reales y generar resultados diversos.

La calidad de los resultados de una simulación depende en gran medida de la calidad de los números aleatorios generados. La precisión y la fiabilidad de los modelos simulados se ven afectadas por la aleatoriedad introducida mediante estos números.

2.Un poco de historia

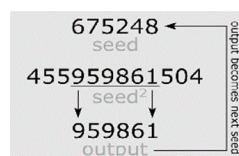
A pesar de que fue en la década del 40 que las primeras computadoras modernas fueron desarrolladas, la simulación ya existía en forma embrionaria aún antes de que la computadora apareciera en escena. Así, por ejemplo, en la segunda mitad del siglo XIX, se realizaban experiencias arrojando agujas al azar sobre una superficie reglada con el fin de estimar el número π . En 1908 W. S. Gosset, bajo el seudónimo de Student, realizaba un muestreo experimental con el fin de descubrir la distribución de un estimador de la correlación en una distribución normal bivariada. En ese momento los números aleatorios se generaban mediante métodos observacionales (mecanismos físicos) tales como tirar un dado, extraer una carta de un mazo o mediante una ruleta.

Dado el esfuerzo que significaba generar números aleatorios cada vez que eran necesarios, parece razonable que se hayan construido tales números y luego tabulado. Tippett (1927) publicó una tabla con 41.600 números aleatorios tomados en forma aleatoria de informes censales. Cada número era uno de los enteros 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 y el usuario tomaba varios de ellos y agregaba un punto decimal para formar un número aleatorio entre 0 y 1. Desde ese momento una serie generadores de números aleatorios fueron propuestos. La primera máquina fue usada en 1939 por Kendall y Babington Smith con el fin de producir una tabla de 100.000 de dígitos aleatorios y en 1955 la RAND Corporation utilizó extensamente una tabla de 1.000.000 dígitos aleatorios que fue obtenida a partir de una ruleta electrónica especialmente diseñada. ERNIE fue una famosa máquina de números aleatorios que fue usada por la lotería británica, es decir la British Premium Savings Bonds Lottery.

Poco después de la aparición de las computadoras, se comenzó a buscar maneras eficientes de obtener números aleatorios, pues aun cuando se podían usar las tablas existentes éste era un recurso

limitado, ya sea por el espacio de memoria necesario como por resultar, en algunos casos, cortas. Si bien máquinas como ERNIE podrían haber trabajado junto con una computadora, una solución en la que la computadora provee todo parecía más satisfactoria. La búsqueda se orientó, entonces, a la producción de números 79 aleatorios usando operaciones aritméticas de una computadora. **John Von Neumann** sugirió en un principio, alrededor de 1946, usar el método del **cuadrado medio**. Su idea era calcular el cuadrado del número aleatorio anterior y tomar los dígitos del medio del número calculado. Así, por ejemplo, si queremos generar un número aleatorio de 10 dígitos y el número anterior es 5772156649 33317792380594909201 el nuevo número será 7923805949. La primera pregunta que cabe hacer es porqué motivo un número generado por este procedimiento que es determinístico, va a resultar aleatorio. La respuesta es que el número no es aleatorio, pero parece serlo, en el sentido en que en una aplicación la relación real entre un número y el siguiente no tiene ningún significado físico. Por lo tanto, el carácter no aleatorio no es una característica indeseable y podría ser que el “cuadrado medio” resultase ser un buen “batido” del número anterior. Es claro, de todas formas, que un mecanismo de esta naturaleza no podría haber reemplazado a ERNIE. Las secuencias de números generadas en forma determinística reciben el nombre de secuencias pseudo-aleatorias o quasi aleatorias, si bien nosotros nos referiremos a ellas como secuencias aleatorias, sobreentendiendo que sólo “parecen” aleatorias. Números aleatorios generados en forma determinística en una computadora funcionan muy bien en muchísimas aplicaciones, a condición de que el método de generación sea bueno.

Volviendo a la propuesta de Von Neumann, ésta no parece ser una buena fuente de números aleatorios. Podría suceder que la secuencia caiga en un ciclo corto de repeticiones, siendo el caso extremo el del cero el cual, si aparece en la secuencia, seguirá repitiéndose siempre. A partir de los años 50 se realizaron diversas experiencias con el método propuesto por Von Neumann. Trabajando con números de 4 dígitos en lugar de 10, G. E. Forsythe probó con 16 números iniciales. Con 12 de ellos terminó con el ciclo 6100, 2100, 4100, 8100, 6100, etc. Y con otras dos terminó en cero. En efecto, $6100^2 = 37210000$ $2100^2 = 4410000$ $4100^2 = 16810000$ $8100^2 = 65610000$ Metrópolis realizó muchas pruebas con los números del “middle square”, en especial con sistemas de números binarios. Mostró que en secuencias de 20 dígitos, hay 13 ciclos diferentes en los que la secuencia puede caer, el más largo de los cuales tiene longitud 142. Estas falencias del “middle-square” son algunas de las consideraciones que debemos hacer ante un generador de números aleatorios.



Para generar números pseudoaleatorios basado en una relación de recurrencia existen varios esquemas, siendo el de **Derrick Henry Lehmer** uno de los más utilizados en las computadoras. En el año 1940 Lehmer desarrollo el primer generador de números pseudo aleatorios denominado **congruencial lineal**.

$$X_{n+1} = (a * X_n + b) \bmod m$$

$$u_n = \frac{X_n}{m}$$

3.Definición de números pseudoaleatorios

Los **números pseudoaleatorios** son generados por medio de una **función determinista**, no aleatoria y aparentan ser aleatorios. Estos números pseudoaleatorios se generan a partir de un **valor inicial** aplicando **iterativamente** la función.

La sucesión de números pseudoaleatorios es sometida a diversos test para medir hasta qué punto se asemeja a una sucesión aleatoria.

Es necesario que la generación de números pseudoaleatorios tengan las siguientes características:

- Que sean uniformemente distribuidos
- Estadísticamente independientes
- Reproducibles
- Periodo Largo
- Generados de manera rápida
- Que no requiera gran capacidad de almacenamiento

La importancia de los números rectangulares (distribuidos uniformemente) radica en su uso para la generación de variables aleatorias requeridas en experimentos de simulación y son generados a través de relación de recurrencia.

Algunos autores han observado que dichos números son pseudoaleatorios (no son puramente aleatorios) por haber sido generados por relaciones de recurrencia apoyadas en reglas deterministas. Sin embargo, esta objeción puede superarse, como ya se mencionó, al menos parcialmente, al tomar un punto de vista un tanto pragmático de que una sucesión puede considerarse aleatoria si satisface un cierto conjunto de pruebas estadísticas de aleatoriedad.

Un número pseudoaleatorio es un valor numérico que se genera mediante un proceso determinístico, usualmente a través de algoritmos matemáticos, y que simula el comportamiento de una secuencia de números aleatorios.

4. Usos y aplicaciones de los números aleatorios

Los generadores de números pseudoaleatorios son ampliamente utilizados en campos tales como el modelado por computadora, estadística, diseño experimental, etc. Algunas de estas secuencias son lo suficientemente aleatorias para ser útiles en estas aplicaciones.

Una de las utilidades principales de los números pseudoaleatorios tiene lugar en los campos de la criptografía.

Asimismo, también destacan su uso en el llamado Método de Montecarlo, con múltiples utilidades, por ejemplo para hallar áreas, volúmenes encerrados en una gráfica y cuyas integrales son muy difíciles de hallar o irresolubles; mediante la generación de puntos basados en estos números, podemos hacer una buena aproximación de la superficie, volumen total, encerrándolo en un cuadrado / cubo, aunque no lo suficientemente buena.

Un campo donde resulta imprescindible, es en la programación de juegos, donde a menudo se necesita disponer de series elegidas al azar. Por ejemplo para crear nubes con patrones diferentes según escenarios. Esto es aún más necesario en aquellos juegos donde el azar es primordial (como juegos donde el azar está implícito en la propia dinámica del juego, por ejemplo juegos de cartas, que necesitan

ser barajadas) o incluso una cuestión que garantice la fiabilidad para dotar al juego de imparcialidad, como en los casos donde en esos juegos se realizan apuestas económicas.

A modo de resumen, se puede sintetizar alguna de las aplicaciones de números pseudoaleatorios para:

- Simular las entradas de aquellas variables aleatorias (no determinísticas).
- Juegos o teoría de decisiones.
- Cálculo numérico (por ejemplo en la resolución de integrales).
- Teoría del muestreo (aquellos casos en los que sea demasiado costoso realizar la muestra).
- Programación (generación de entradas para realizar las pruebas de los algoritmos y programas).
- Generadores de números aleatorios en lenguaje C, C++, Delphi o Java, utilizan métodos congruenciales.
- lineales (como se verá a continuación).
- Muestreo: con el fin de seleccionar una submuestra de una población.
- Programación: la generación de valores aleatorios puede ser útil para poner a prueba la efectividad de un algoritmo. También son útiles en criptología.

5. Generación de números pseudoaleatorios

Para poder realizar una simulación que incluya variabilidad dentro de sus eventos, es preciso generar una serie de números que sean aleatorios por sí mismos, y que su aleatoriedad se extrapole al modelo de simulación que se está construyendo. Los números aleatorios en la simulación de sistemas juegan un papel relevante. Así, una de las primeras tareas que es necesario llevar a cabo consiste en determinar si los números que se utilizarán para correr la simulación son realmente aleatorios o no. Por desgracia, precisar lo anterior con absoluta certidumbre resulta muy complicado, ya que para ello tendríamos que generar un número infinito de valores que nos permitiera comprobar la inexistencia de correlaciones entre ellos. Esto sería muy costoso y tardado, además volvería impráctico el uso de la simulación aun con las computadoras más avanzadas.

A pesar de lo anterior, podemos asegurar que el conjunto de números que utilizaremos en una simulación se comporta de manera muy similar a un conjunto de números totalmente aleatorios; por ello es que se les denomina **números pseudoaleatorios**. La gran mayoría de las aplicaciones tiene varios generadores de números pseudoaleatorios que pueden generar un conjunto muy grande de números sin mostrar correlación entre ellos. Se discuten aquí algunos de los métodos de generación de números pseudoaleatorios, y precisaremos qué características deben tener para emplearlos como una fuente confiable de variabilidad dentro de los modelos. Asimismo, se mostrarán algunas de las pruebas más comunes para comprobar qué tan aleatorios son los números obtenidos con dichos generadores.

Para realizar una simulación se requieren **números aleatorios en el intervalo (0,1)**, a los cuales se hará referencia como:

$$\mathbf{r_i} = \{r_1, r_2, r_3, \dots, r_n\}$$

La secuencia contiene **n números**, todos ellos **diferentes**.

Los **r_i** constituyen la parte medular de la simulación de procesos estocásticos, y por lo regular, se usan para generar el comportamiento de variables aleatorias, tanto continuas como discretas. Debido a que no es posible generar números realmente aleatorios, consideramos los **r_i** como números

pseudoaleatorios generados por medio de algoritmos determinísticos que requieren parámetros de arranque.

Para simular el comportamiento de una o más variables aleatorias es necesario contar con un conjunto suficientemente grande de r_i .

En principio consideraremos métodos para generar números con **distribución uniforme** en el intervalo **(0,1)**. Esto podemos lograrlo generando enteros X_n entre **0** y un número natural **m** y luego tomando la fracción:

$$r_n = \frac{x_n}{m}$$

Para comprender porque un generador puede ser considerado mejor que otro es necesario saber cómo operan los mismos. El método más común es generar el siguiente número a partir de los últimos números generados:

$$x_n = f(x_{n-1}, x_{n-2}, x_{n-3}, \dots, x_2, x_1)$$

A modo de ejemplo, se puede analizar la función **$x_n = (5x_{n-1} + 1) \bmod 16$** .

Se comienza con $x_0 = 5$; los primeros 16 números diferentes generados a partir de esta semilla son: **10, 3, 0, 1, 6, 15, 12, 13, 2, 11, 8, 9, 14, 7, 4, 5**. Si se continúa con el proceso de generación de números, se repite la secuencia.

Las x son enteros entre 0 y 15. Si se calcula $x / 16$ se obtiene una secuencia de números aleatorios entre 0 y 1: 0.6250; 0.1875; 0, ...

Si se conoce la función f se puede generar la secuencia en cualquier momento a partir de un valor x_0 conocido.

El valor x_0 usado para comenzar la secuencia de números pseudoaleatorios se llama SEMILLA.

**f es DETERMINÍSTICA.
Dada la semilla se puede predecir con probabilidad 1 los números de la secuencia.**

Sin embargo, los números son aleatorios en el sentido de que pasan pruebas estadísticas de aleatoriedad y por esto son llamados pseudoaleatorios. En muchos casos se prefieren estos números en vez de los completamente aleatorios ya que es necesario repetir las secuencias en distintos experimentos. Si deseamos otra secuencia simplemente cambiamos la semilla.

En el ejemplo la secuencia tiene un ciclo y la longitud del ciclo es 16.

La mayoría de los métodos comienzan con una **semilla**. A este número se le aplica un determinado **procedimiento**, se encuentra el primer **número random**. Usando este número como **entrada**, se repite el **procedimiento** para lograr un **próximo número random**.

6. Generador Congruencial Lineal

Para generar números pseudoaleatorios basado en una relación de recurrencia existen varios esquemas, siendo el de Derrick Henry Lehmer uno de los más utilizados en las computadoras. En el año de 1940 Lehmer desarrollo el primer generador de números pseudo aleatorios denominado congruencial lineal.

El **generador congruencial lineal** genera una secuencia de números pseudo aleatorios en la cual el próximo número pseudo aleatorio es calculado a partir del último numero pseudoaleatorio generado. Es decir x_{n+1} se deriva del numero x_n . Opera a través de una fórmula recursiva que produce secuencias de enteros que parecen ser aleatorias. El proceso implica multiplicar el **número generado anteriormente** por una **constante a**, sumarle otra **constante b**, y luego tomar el **resto de la división** de este resultado por un **módulo m**.

El **GENERADOR CONGRUENCIAL LINEAL** utiliza la relación de recurrencia:

$$x_{n+1} = (ax_n + b) \bmod m \quad 0 \leq x_n < m \quad \forall n$$

Se distinguen cuatro elementos:

x_0 valor inicial o **semilla** - $x_0 > 0$.
a elemento **multiplicador** - $0 < a < m$.
b incremento o **constante aditiva** - $0 \leq b < m$.
m **módulo**

$$\begin{array}{r} ax_n + b \\ \hline m \\ \text{cociente} \end{array}$$

resto= x_{n+1}

Los generadores congruenciales lineales más populares son el congruencial mixto y el congruencial multiplicativo.

El número aleatorio μ_i para cada valor de n (0, 1, 2, 3...) es: $\mu_i = x_i / m$

El **PERÍODO** es la subcadena de números x_i , dentro de la serie generada, en la que no hay repeticiones de números.

La **LONGITUD DEL PERÍODO** es el número de elementos distintos en la subcadena.

Cuando la longitud del período coincide con el módulo, se dice que el generador es de **CICLO COMPLETO**.

La repetición de números en la serie puede ser aleatoria, pero dado el método utilizado para la generación de las mismas, en el momento en el que se repite un valor ya empieza a repetirse todo el período, por lo que interesan métodos que garanticen longitudes de periodo grandes.

i. Generador Congruencial Lineal Mixto

$$x_{n+1} = ax_n + b \pmod{m}$$

El incremento o constante aditiva **b** es diferente de 0.

Ejemplo

Semilla x_0	8
multiplicador a	5
incremento b	7
módulo m	16

Longitud período 16
Ciclo completo

i	x_{i-1}	x_i	
1	8	0,5	No se repite
2	15	0,938	No se repite
3	2	0,125	No se repite
4	1	0,063	No se repite
5	12	0,750	No se repite
6	3	0,188	No se repite
7	6	0,375	No se repite
8	5	0,313	No se repite
9	0	0,000	No se repite
10	7	0,438	No se repite
11	10	0,625	No se repite
12	9	0,563	No se repite
13	4	0,250	No se repite
14	11	0,688	No se repite
15	14	0,875	No se repite
16	13	0,813	No se repite
17	8	0,500	Se Repite

Semilla x_0	8
multiplicador a	5
incremento b	4
módulo m	16

Longitud Período 4
Ciclo incompleto

i	x_{i-1}	x_i	
1	8	0,5	No se repite
2	12	0,750	No se repite
3	0	0,000	No se repite
4	4	0,250	No se repite
5	8	0,500	Se Repite

* Elección de parámetros adecuados

En la década de los 60, el Dr. Donald Knuth propuso una regla para la selección de los parámetros para obtener una buena secuencia de números generada por el método congruencial lineal mixto. Esta regla se enfoca en elegir cuidadosamente estos parámetros para garantizar propiedades estadísticas deseables en la secuencia de números pseudoaleatorios. Al seleccionar apropiadamente estos valores, se puede mejorar la uniformidad y la aleatoriedad de la secuencia generada, lo que es crucial para aplicaciones que requieren números pseudoaleatorios de alta calidad.

Hoy en día los generadores más eficientes tienen $m = 2^{32}$ para computadoras de 32 bits y $m = 2^{64}$ para computadoras de 64 bits, debido a que es el valor máximo para calcular la operación de modulo debido a truncamiento.

El módulo m debe ser grande. Dado que los x están entre 0 y $m-1$, el periodo nunca puede ser mayor que m . Si $m = 2^5$ y el ciclo es completo, la serie generada tendrá los números 0, 1, 2, 3 ...31.

Una cuestión de interés es cómo elegir los parámetros del generador de forma que este tenga ciclo completo. Para que el computo de m sea **eficiente**, m debe ser una potencia de p^d , p es la base del sistema (decimal, binario, hexadecimal).

El **teorema de Hull y Dobell**, establece las **condiciones necesarias y suficientes para que un generador congruencial mixto alcance un ciclo completo**, es decir, que genere todas las secuencias posibles en su período antes de repetirse. El teorema es fundamental para garantizar la aleatoriedad y la eficacia de los generadores congruenciales en la generación de números pseudoaleatorios.

Un generador congruencial mixto tiene periodo completo si y sólo si se cumplen las siguientes condiciones:

- m y b son primos entre sí (no tienen factores comunes excepto el 1).
- Si q es un número primo que divide a m , entonces q divide a $(a-1)$. Aplicar para todo q primo que divide a m .
- Si 4 divide a m , entonces 4 divide a $(a-1)$.

Ejemplo

Semilla x_0	8
multiplicador a	5
incremento b	7
módulo m	16

Longitud período 16
Ciclo completo

m y b son primos entre sí

16 y 7 son primos entre sí

Si q es un número primo que divide a m , entonces q divide a $(a-1)$

2 divide a 16, también divide a 4 (si $a=5 \Rightarrow a-1=4$)

SE DEBE EVALUAR ESTA CONDICION CON TODOS LOS NÚMEROS
Q PRIMOS POSIBLES QUE DIVIDAN AL MÓDULO

Si 4 divide a m , entonces 4 divide a $(a-1)$

4 divide a 16, también divide a 4 (si $a=5 \Rightarrow a-1=4$)

Semilla x_0	8
multiplicador a	5
incremento b	4
módulo m	16

Longitud Período 4
Ciclo incompleto

m y b son primos entre sí

16 y 4 NO son primos entre sí - **NO SE CUMPLE LA CONDICION**

Si q es un número primo que divide a m , entonces q divide a $(a-1)$

2 divide a 16, también divide a 4 (si $a=5 \Rightarrow a-1=4$)

Si 4 divide a m , entonces 4 divide a $(a-1)$

4 divide a 16, también divide a 4 (si $a=5 \Rightarrow a-1=4$)

SI UNA CONDICION NO SE CUMPLE, ES SUFICIENTE PARA ASEGURAR
QUE EL GENERADOR NO ES DE CICLO COMPLETO

ii. Generador Congruencial Lineal Multiplicativo

$$x_{n+1} = ax_n \mod m$$

El incremento o constante aditiva b es igual a 0.

Estos generadores presentan la ventaja sobre los congruenciales mixtos de ser más rápidos al tener que realizar menos operaciones en el cálculo de los elementos. Sin embargo, la longitud de periodo que se alcanza en las series generadas por los congruenciales multiplicativos son menores que la alcanzadas por los congruenciales mixtos.

Los generadores congruenciales multiplicativos **NO producen secuencias de **CICLO COMPLETO****

* Elección de parámetros adecuados

Se observa que si se elige un módulo donde m es primo, se obtienen mejores resultados. Esto llevó a que los **métodos congruenciales lineales multiplicativos**, puedan desarrollar generadores con **longitud de período $t=m-1$** .

El **teorema de Hutchinson-Lehmer** (1966) plantea:

Sea t la longitud de un ciclo maximal igual a $(m-1)$ de un generador congruencial multiplicativo, con módulo m primo. Se verifica:

- $t = m - 1$.
- t es de longitud maximal si y sólo si a es una raíz primitiva de m si :
 - $a \neq 0$
 - No existe ningún factor primo p de $(m - 1)$ tal que :

$$1 = a^{\frac{(m-1)}{p}} \mod m$$

Aplicar para todo p primo que divida a $(m-1)$.

Ejemplo

Determinar cual de los dos generadores congruenciales multiplicativos logran el ciclo maximal t posible.

1. $x_n = 3x_{n-1} \mod 31$
2. $x_n = 4x_{n-1} \mod 31$

En los dos casos, el módulo es 31, número primo. Si $m=31 \Rightarrow t=30$

Se debe evaluar el teorema para dos valores posibles de a , 3 y 4.

- Si $t = m - 1 = 30$, entonces $m=31$ es primo.
- Si $m=31$ es primo, entonces $t = 30$ divide a $m - 1$.
- Si $m=31$ es primo, entonces t es $(m - 1) = 30$ si y sólo si a es una raíz primitiva de m ; a es una raíz primitiva de m si :
 - $a \neq 0$. Se cumple porque a puede tomar el valor 3 o 4.
 - Se debe demostrar que no existe ningún factor primo p de 30 tal que:

$$1 = a^{\frac{30}{p}} \mod 31$$

Los factores primos p de 30 son: 2, 3, 5. El análisis debe realizarse con todos los factores primos de $m-1$

$$x_n = 3x_{n-1} \mod 31$$

$$x_n = 4x_{n-1} \mod 31$$

p : primo de $(m-1) = 30$	$a^{(m-1)/p}$	$a^{(m-1)/p}$	$a^{(m-1)/p} \mod m$	Condición
2	3^{15}	14.348.907	30	Cumple principio
3	3^{10}	59.049	25	Cumple principio
5	3^6	729	16	Cumple principio

p : primo de $(m-1) = 30$	$a^{(m-1)/p}$	$a^{(m-1)/p}$	$a^{(m-1)/p} \mod m$	Condición
2	4^{15}	1.073.741.824	1	No Cumple principio
3	4^{10}	1.048.576	1	No Cumple principio
5	4^6	4.096	4	Cumple principio

Se concluye que el generador $x_n = 3x_{n-1} \mod 31$ logra longitud de ciclo 30.

Se generan números aleatorios con semilla 5 en cada caso.

$$x_n = 3x_{n-1} \bmod 31$$

Longitud Período 30					
i	xi-1	ui	i	xi-1	ui
1	5	0,161	17	16	0,516
2	15	0,484	18	17	0,548
3	14	0,452	19	20	0,645
4	11	0,355	20	29	0,935
5	2	0,065	21	25	0,806
6	6	0,194	22	13	0,419
7	18	0,581	23	8	0,258
8	23	0,742	24	24	0,774
9	7	0,226	25	10	0,323
10	21	0,677	26	30	0,968
11	1	0,032	27	28	0,903
12	3	0,097	28	22	0,710
13	9	0,290	29	4	0,129
14	27	0,871	30	12	0,387
15	19	0,613	31	5	0,161
16	26	0,839			

$$x_n = 4x_{n-1} \bmod 31$$

Longitud Período 5		
i	xi-1	ui
1	5	0,161
2	20	0,645
3	18	0,581
4	10	0,323
5	9	0,290
6	5	0,161

Si $m-1=30$, se puede expresar $m-1$ en función de sus divisores: $30 = 1 * 2 * 3 * 5$. La longitud de un generador con módulo 31 puede ser 1, 2, 3, 5, 6, 10, 15 y 30.

7. Generador por Método del Cuadrado Medio

Es un método propuesto por los matemáticos **John von Neumann** y **Nicholas Metropolis**, siendo utilizado para la generación de números pseudoaleatorios. La sucesión de números básicamente se obtienen a partir de recurrencia.

El procedimiento consiste en que cada número de una sucesión es producido tomando los dígitos medios de un número obtenido mediante la elevación al cuadrado, de acuerdo al siguiente detalle:

1. Se define la **semilla**.
2. Se eleva la semilla al **cuadrado**.

3. Se toma la **parte central** del número resultante. **Si no es posible determinar la parte central, se completa el número agregando ceros al principio o al final**. La cantidad de números que se extraen depende de la cantidad de dígitos que tendrá el número aleatorio.

4. Para obtener números aleatorios entre 0 y 1, el resultado se debe **normalizar**. Si los números son de dos dígitos se normaliza dividiendo por 100, si son de tres dígitos por 1.000 y así sucesivamente. Esto significa que el **número obtenido en el paso anterior se divide por 10^n** , donde n representa la cantidad de dígitos del número aleatorio.

5. La parte **central del número obtenido en el paso 3 se transforma en la semilla**. El procedimiento se repite, volviendo al paso 2.

Algunas desventajas del método:

- Tiene una fuerte tendencia a degenerar a cero rápidamente.
- Los números generados pueden repetirse cíclicamente después de una secuencia corta.

Ejemplo

Generar números aleatorios por el método del cuadrado medio. Considere semilla igual a 2.009. Los números deben tener 4 dígitos.

Semilla 2009															
Se generan números con 4 dígitos															
Para normalizar cada valor de x se divide por 10^4 = 10.000															
i	xi	(xi)^2	Longitud número	Completa con 0	Parte Central	Número Aleatorio		i	xi	(xi)^2	Longitud número	Completa con 0	Parte Central	Número Aleatorio	
1	2009	4036081	7	40360810	3608	0,2009	Sin repetir	18	4739	22458121	8	22458121	4581	0,4739	Sin repetir
2	3608	13017664	8	13017664	176	0,3608	Sin repetir	19	4581	20985561	8	20985561	9855	0,4581	Sin repetir
3	176	30976	5	30976000	9760	0,0176	Sin repetir	20	9855	97121025	8	97121025	1210	0,9855	Sin repetir
4	9760	95257600	8	95257600	2576	0,976	Sin repetir	21	1210	1464100	7	14641000	6410	0,121	Sin repetir
5	2576	6635776	7	66357760	3577	0,2576	Sin repetir	22	6410	41088100	8	41088100	881	0,641	Sin repetir
6	3577	12794929	8	12794929	7949	0,3577	Sin repetir	23	881	776161	6	77616100	6161	0,0881	Sin repetir
7	7949	63186601	8	63186601	1866	0,7949	Sin repetir	24	6161	37957921	8	37957921	9579	0,6161	Sin repetir
8	1866	3481956	7	34819560	8195	0,1866	Sin repetir	25	9579	91757241	8	91757241	7572	0,9579	Sin repetir
9	8195	67158025	8	67158025	1580	0,8195	Sin repetir	26	7572	57335184	8	57335184	3351	0,7572	Sin repetir
10	1580	2496400	7	24964000	9640	0,158	Sin repetir	27	3351	11229201	8	11229201	2292	0,3351	Sin repetir
11	9640	92929600	8	92929600	9296	0,964	Sin repetir	28	2292	5253264	7	52532640	5326	0,2292	Sin repetir
12	9296	86415616	8	86415616	4156	0,9296	Sin repetir	29	5326	28366276	8	28366276	3662	0,5326	Sin repetir
13	4156	17272336	8	17272336	2723	0,4156	Sin repetir	30	3662	13410244	8	13410244	4102	0,3662	Sin repetir
14	2723	7414729	7	74147290	1472	0,2723	Sin repetir	31	4102	16826404	8	16826404	8264	0,4102	Sin repetir
15	1472	2166784	7	21667840	6678	0,1472	Sin repetir	32	8264	68293696	8	68293696	2936	0,8264	Sin repetir
16	6678	44595684	8	44595684	5956	0,6678	Sin repetir	33	2936	8620096	7	86200960	2009	0,2936	Sin repetir
17	5956	35473936	8	35473936	4739	0,5956	Sin repetir	34	2009	4036081	7	40360810	3608	0,2009	Se repite

8. Bibliografía

Simulación y análisis de sistemas con Promodel. Segunda edición. Eduardo García Dunna, Heribete García Reyes, Leopoldo Eduardo Cárdenas Barrón.

Simulación. Un enfoque práctico. Raúl Coss_Bu.

Simulación. Ing. David Sergio Castellón Domínguez. Docente del departamento de Sistemas y Computación.
Instituto Tecnológico de Piedras Negras.

<https://colab.research.google.com>